

Complementos de Investigação Operacional

5.1 Programação dinâmica: princípio de Bellman e caminho mais curto

José Firmino Aguilar Madeira

Decidir por etapas

Muitos problemas são uma **sequência de decisões**: que estrada seguir, que produto carregar, quantas pessoas ter em cada semana.

Enumerar todas as sequências cresce depressa demais. A **programação dinâmica** (PD) resolve o problema **de trás para a frente**, uma etapa de cada vez, reaproveitando o que já calculou.

Princípio da otimalidade (Bellman, 1957)

Uma política ótima tem a propriedade de, quaisquer que sejam o estado e a decisão iniciais, as decisões seguintes formarem uma política ótima a partir do estado que resulta da primeira decisão.

Ingredientes

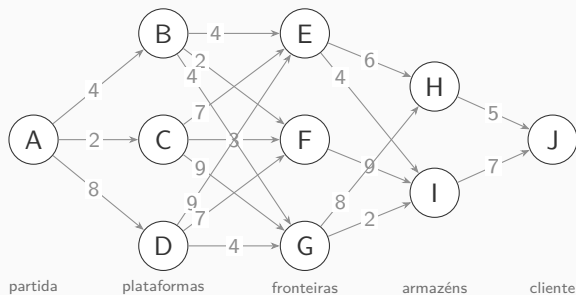
- **etapas** $n = 1, 2, \dots, N$;
- **estado** s : o que é preciso saber para decidir daqui para a frente;
- **decisão** x em cada etapa;
- **recursão**, com novo estado $s' = T(s, x)$:

$$f_n(s) = \min_x \{c_n(s, x) + f_{n+1}(T(s, x))\}.$$

Nos três problemas deste capítulo, o novo estado é a própria decisão (o nó seguinte, a capacidade que sobra, os operários desta semana).

Neste capítulo: caminho mais curto (5.1), mochila (5.2) e gestão de mão-de-obra (5.3).

O exemplo: levar uma carga de A a J



Uma transportadora leva uma carga de A até ao cliente J, passando por uma plataforma logística (B, C, D), um ponto de fronteira (E, F, G) e um armazém de distribuição (H, I).

Nos arcos: custo do troço, em centenas de €.

Pergunta: qual é o percurso mais barato?

Há 15 percursos possíveis.

Escolher sempre o troço mais barato? Não.

Regra gulosa (míope): em cada nó, seguir o arco mais barato.

- $A \rightarrow C$ (2), $C \rightarrow F$ (3), $F \rightarrow I$ (9), $I \rightarrow J$ (7).
- Total: $2 + 3 + 9 + 7 = 21$.

O arco barato $A \rightarrow C$ leva a uma zona da rede onde tudo o que vem a seguir é caro. Uma decisão só pode ser avaliada **com as consequências**.

A ideia da PD

Começar pelo fim. Para cada nó, calcular o custo mínimo de lá até J, $f(\text{nó})$. Quando se chega a A, já se sabe tudo.

$$f(i) = \min_j \{c_{ij} + f(j)\}, \quad f(J) = 0.$$

Recursão para trás

Etapas 4 (armazéns): $f(H) = 5$, $f(I) = 7$.

Etapas 3 (fronteiras)

nó	via H	via I	f	decisão
E	$6 + 5 = 11$	$4 + 7 = 11$	11	H ou I
F	—	$9 + 7 = 16$	16	I
G	$8 + 5 = 13$	$2 + 7 = 9$	9	I

Etapas 1 (partida)

nó	via B	via C	via D	f	decisão
A	$4 + 13 = 17$	$2 + 18 = 20$	$8 + 13 = 21$	17	B

Etapas 2 (plataformas)

nó	via E	via F	via G	f	dec.
B	15	18	13	13	G
C	18	19	18	18	E ou G
D	20	23	13	13	G

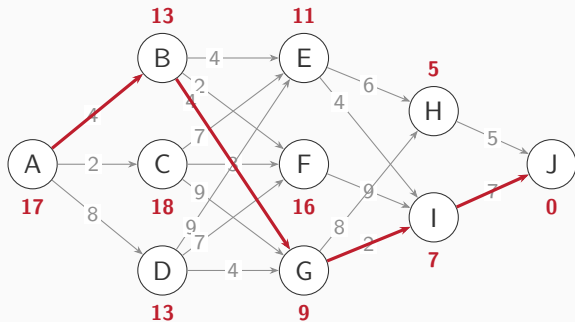
(por exemplo, B via G: $4 + f(G) = 4 + 9 = 13$)

Ler a solução para a frente

$A \rightarrow B$ (decisão em A) $\rightarrow G$ (em B) $\rightarrow I$ (em G) $\rightarrow J$.

Custo mínimo: **17** (1700 €), contra 21 da regra gulosa.

O percurso ótimo



a vermelho: $f(\text{nó})$, custo mínimo até J

$A \rightarrow B \rightarrow G \rightarrow I \rightarrow J$, custo 17.

Os valores f dão mais do que a resposta: dizem o melhor caminho **a partir de qualquer nó**. Se a carga já estiver em C (por exemplo, por um desvio), o melhor é seguir por E ou por G, com custo 18.

É esta “política para todos os estados” que distingue a PD de uma simples procura do caminho.

Quanto se poupa?

Neste exemplo: enumerar os 15 percursos são $15 \times 3 = 45$ somas; a PD faz uma soma por arco: 19.

Em geral: com N etapas e k nós por etapa,

- enumerar: k^N percursos;
- PD: cerca de $N k^2$ somas.

N	k	percursos	somas da PD
4	3	81	36
10	10	10^{10}	1000
20	10	10^{20}	2000

Para redes sem etapas

Em redes gerais com custos não negativos (ruas de uma cidade, rede elétrica), usa-se normalmente o algoritmo de **Dijkstra**.

Partilha a mesma ideia de subestrutura ótima, embora não seja a recursão por etapas usada aqui. Está por trás dos sistemas de navegação.

A PD exige que o problema se decomponha em etapas e que o estado resuma tudo o que interessa do passado (propriedade de Markov).

```
arcos = {'A': {'B': 4, 'C': 2, 'D': 8},
        'B': {'E': 4, 'F': 2, 'G': 4},
        'C': {'E': 7, 'F': 3, 'G': 9},
        'D': {'E': 9, 'F': 7, 'G': 4},
        'E': {'H': 6, 'I': 4}, 'F': {'I': 9},
        'G': {'H': 8, 'I': 2}, 'H': {'J': 5}, 'I': {'J': 7}}
ordem = ['H', 'I', 'E', 'F', 'G', 'B', 'C', 'D', 'A'] # do fim
f, dec = {'J': 0}, {}
for i in ordem:
    dec[i] = min(arcos[i], key=lambda j: arcos[i][j] + f[j])
    f[i] = arcos[i][dec[i]] + f[dec[i]]
no, caminho = 'A', ['A']
while no != 'J':
    no = dec[no]; caminho.append(no)
print(f['A'], caminho) # 17 ['A', 'B', 'G', 'I', 'J']
```

Cada nó é calculado uma vez, depois dos seus sucessores.

Para redes grandes:

```
networkx.shortest_path(G, 'A', 'J',
weight='custo').
```

Ficheiro completo (caminho mais curto por etapas):

```
code/python/05_dynamic_programming/
05_1_bellman_shortest_path/
caminho.py
```

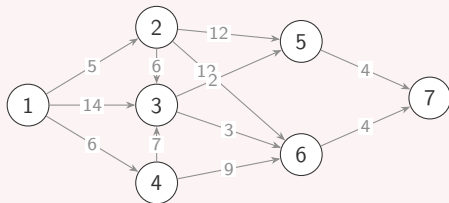
Exemplo 1 — caminho mais curto do nó 1 ao nó 7

Distâncias: $d(1,2) = 5$, $d(1,3) = 9$, $d(1,4) = 8$,
 $d(2,5) = 10$, $d(2,6) = 17$, $d(3,5) = 4$, $d(3,6) = 10$,
 $d(4,5) = 9$, $d(4,6) = 9$, $d(5,7) = 19$, $d(6,7) = 9$.

Exemplo 2 — caminho mais curto do nó 1 ao nó 7

$d(1,2) = 7$, $d(1,3) = 8$, $d(1,4) = 9$, $d(2,5) = 12$,
 $d(3,5) = 8$, $d(3,6) = 9$, $d(4,5) = 7$, $d(4,6) = 13$,
 $d(5,7) = 9$, $d(6,7) = 6$.

Exemplo 3 — caminho mais curto do nó 1 ao nó 7



Em cada caso: defina etapas e estados, faça a recursão para trás e indique todos os percursos ótimos.

O QUE DEVE FICAR — 5.1

1. PD: decompor em **etapas**, definir o **estado**, escrever a **recursão** e resolver do fim para o início.
2. Princípio de Bellman: a partir de qualquer estado, o resto de uma política ótima é ótimo.
3. Caminho mais curto: $f(i) = \min_j \{c_{ij} + f(j)\}$, $f(\text{destino}) = 0$; ler a solução para a frente.
4. A regra gulosa pode falhar (21 contra 17). A PD avalia cada decisão com as suas consequências.
5. Os valores f dão a melhor decisão a partir de *qualquer* estado.

A seguir: 5.2 O problema da mochila

Referências Lista completa: *Bibliografia da UC*.

- Hillier, F. S., Lieberman, G. J., *Introduction to Operations Research*, 11.^a ed., McGraw-Hill, 2021 — Dynamic Programming (o problema da diligência).
- Bellman, R., *Dynamic Programming*, Princeton University Press, 1957.
- Kasana, H. S., Kumar, K. D., *Introductory Operations Research*, Springer, 2004 — cap. 12.
- Taha, H. A., *Operations Research: An Introduction*, 10.^a ed., Pearson, 2017 — programação dinâmica determinística.

Recursos

- Lista de exercícios 5. Trabalho prático n.^o 2: criar e resolver 2 problemas de programação dinâmica.